

HƯỚNG DẪN THỰC HÀNH

ORACLE PL/SQL - BUỔI 3

Block PL/SQL • Cursor • Procedure • Function • Trigger

**Schema: Quan ly Khoa Hoc (COURSE, CLASS, STUDENT,
ENROLLMENT, INSTRUCTOR, GRADE)**

Nội dung: PL/SQL Block - Transaction - Cursor - Procedure - Function -
Trigger

Gom 5 bài với mức độ từ cơ bản đến nâng cao

PHẦN 1: HỆ THỐNG KIẾN THỨC PL/SQL ORACLE

PL/SQL (Procedural Language / SQL) là ngôn ngữ lập trình thủ tục của Oracle, mở rộng SQL với các cấu trúc lập trình như biến, điều kiện, vòng lặp, cursor, thủ tục, hàm và trigger.

1.1. Cấu Trúc Block PL/SQL

Mọi chương trình PL/SQL đều được tổ chức thành các block. Mỗi block có 3 phần:

```
DECLARE
    -- Phân khai bao biến, hằng số, con trỏ (tùy chọn)
    v_ten      VARCHAR2(25);
    v_so       NUMBER := 0;
    c_hang     CONSTANT NUMBER := 100;
BEGIN
    -- Phân xử lý chính (bắt buộc)
    v_ten := 'Oracle';
    DBMS_OUTPUT.PUT_LINE('Xin chào: ' || v_ten);
```

```

EXCEPTION
    -- Xu ly ngoai le (tuy chon)
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Khong tim thay du lieu!');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Loi: ' || SQLERRM);
END;
/

```

• **Bật hiển thị output:**

Trước khi chạy PL/SQL, phải bật chế độ hiển thị: SET SERVEROUTPUT ON;
Nếu không sẽ không thấy kết quả của DBMS_OUTPUT.PUT_LINE.

1.2. Khai Báo Biến Và Kiểu Dữ Liệu

Cú pháp khai báo	Ý nghĩa	Ví dụ
<code>ten_bien kieu_du_lieu;</code>	Biến chưa có giá trị (NULL)	<code>v_name VARCHAR2(25);</code>
<code>ten_bien kieu := gia_tri;</code>	Biến có giá trị khởi tạo	<code>v_count NUMBER := 0;</code>
<code>ten_bien bang%TYPE;</code>	Kiểu giống kiểu cột của bảng (khuyến dùng)	<code>v_name student.firstname%TYPE;</code>
<code>ten_bien bang%ROWTYPE;</code>	Kiểu bản ghi - chứa toàn bộ một hàng của bảng	<code>v_row student%ROWTYPE;</code>
<code>CONSTANT hang_so kieu := gt;</code>	Hằng số - không thể thay đổi sau khi khai báo	<code>c_rate CONSTANT NUMBER := 0.05;</code>

Câu lệnh SELECT INTO - gán kết quả truy vấn vào biến:

```

SELECT ten_cot INTO ten_bien FROM ten_bang WHERE dieu_kien;
-- Ví dụ:
SELECT firstname, lastname INTO v_fname, v_lname
FROM student WHERE studentid = v_id;

```

❖ **Lưu ý:** SELECT INTO phải trả về **ĐÚNG 1 hàng**. Nếu không có hàng → NO_DATA_FOUND.
Nếu nhiều hơn 1 hàng → TOO_MANY_ROWS.

1.3. Biến Nhập Từ Người Dùng - Substitution Variable

Trong SQL*Plus, dùng ký hiệu & để nhận giá trị từ bàn phím người dùng:

```
-- Người dùng sẽ được hỏi nhập giá trị khi chạy  
DECLARE  
    v_id NUMBER := &ma_sinh_vien; -- Hỏi nhập số  
    v_ten VARCHAR2(25) := '&ten'; -- Hỏi nhập chuỗi  
BEGIN  
    ...  
END;  
/
```

1.4. Cấu Trúc Điều Khiển

IF - ELSIF - ELSE

```
IF điều_kiện_1 THEN  
    -- xử lý 1  
ELSIF điều_kiện_2 THEN  
    -- xử lý 2  
ELSE  
    -- xử lý mặc định  
END IF;
```

CASE

```
-- Dạng đơn giản (so sánh giá trị)  
CASE biểu_thức  
    WHEN giá_trị_1 THEN xử_ly_1;
```

```
    WHEN gia_tri_2 THEN xu_ly_2;

    ELSE xu_ly_mac_dinh;

END CASE;

-- Dang tim kiem (dieu kien logic)

CASE

    WHEN diem >= 90 THEN v_grade := 'A';

    WHEN diem >= 80 THEN v_grade := 'B';

    WHEN diem >= 70 THEN v_grade := 'C';

    WHEN diem >= 50 THEN v_grade := 'D';

    ELSE v_grade := 'F';

END CASE;
```

Vòng Lặp LOOP, WHILE, FOR

-- LOOP co ban (can EXIT de thoat)

```
LOOP

    EXIT WHEN dieu_kien;

    -- xu ly

END LOOP;
```

-- WHILE LOOP

```
WHILE dieu_kien LOOP

    -- xu ly

END LOOP;
```

-- FOR LOOP (tu dong tang i)

```
FOR i IN 1..10 LOOP
```

```
    DBMS_OUTPUT.PUT_LINE(i);
```

```
END LOOP;
```

1.5. Quản Lý Giao Dịch — Transaction, COMMIT, ROLLBACK, SAVEPOINT

Trong PL/SQL, các thao tác DML (INSERT/UPDATE/DELETE) tạo thành một giao dịch.

Bạn có thể kiểm soát việc lưu hoặc hủy thay đổi:

Lệnh	Ý nghĩa	Ví dụ
COMMIT;	Xác nhận và lưu vĩnh viễn tất cả thay đổi từ lần COMMIT trước	COMMIT;
ROLLBACK;	Hủy toàn bộ thay đổi chưa COMMIT, quay về trạng thái ban đầu	ROLLBACK;
SAVEPOINT ten_sp;	Tạo điểm đánh dấu trong giao dịch để ROLLBACK một phần	SAVEPOINT sp_a;
ROLLBACK TO ten_sp;	Hủy thay đổi đến điểm Savepoint đã đánh dấu (không hủy toàn bộ)	ROLLBACK TO sp_a;

```
BEGIN
```

```
    INSERT INTO bang VALUES (...);
```

```
    SAVEPOINT sp_a;          -- Danh dau diem A
```

```
    INSERT INTO bang VALUES (...);
```

```
    SAVEPOINT sp_b;          -- Danh dau diem B
```

```
    ROLLBACK TO sp_a;        -- Hủy tu diem A den hien tai (giu lai INSERT dau)
```

```
COMMIT;          -- Lưu vĩnh viễn phân con lại  
END;  
/
```

1.6. Sequence - Tạo Số Tự Động

Sequence là đối tượng Oracle tạo ra các giá trị số tuần tự (thường dùng làm khóa chính):

-- Tạo sequence

```
CREATE SEQUENCE ten_seq  
  START WITH 1      -- Bắt đầu từ 1  
  INCREMENT BY 5    -- Bước tăng là 5  
  NOCACHE           -- Không cache (tùy chọn)  
  NOCYCLE;          -- Không lặp lại (tùy chọn)
```

-- Sử dụng sequence

```
ten_seq.NEXTVAL  -- Lấy giá trị tiếp theo (tăng lên)  
ten_seq.CURRVAL  -- Lấy giá trị hiện tại (đã lấy qua NEXTVAL rồi)
```

-- Ví dụ trong INSERT:

```
INSERT INTO bang (id, name) VALUES (ten_seq.NEXTVAL, 'ABC');
```

1.7. Cursor - Xử Lý Tập Kết Quả Nhiều Hàng

Cursor là con trỏ trỏ vào tập kết quả của một câu SELECT trả về nhiều hàng. Có 2 loại:

Cursor tường minh (Explicit Cursor) - đầy đủ 4 bước**DECLARE**`-- Buoc 1: Khai bao cursor``CURSOR cur_ten IS` `SELECT cot1, cot2 FROM bang WHERE dieu_kien;``-- Khai bao cursor co doi so (tham so)``CURSOR cur_co_ds (p_id NUMBER) IS` `SELECT * FROM bang WHERE id = p_id;``v_cot1 bang.cot1%TYPE;``v_cot2 bang.cot2%TYPE;`**BEGIN**`-- Buoc 2: Mo cursor``OPEN cur_ten;``-- Buoc 3: Doc tung hang``LOOP` `FETCH cur_ten INTO v_cot1, v_cot2;` `EXIT WHEN cur_ten%NOTFOUND; -- Thoat khi het hang` `DBMS_OUTPUT.PUT_LINE(v_cot1 || ' - ' || v_cot2);``END LOOP;``-- Buoc 4: Dong cursor``CLOSE cur_ten;`

```
END;
```

```
/
```

Cursor FOR LOOP - cách viết gọn hơn (tự động OPEN/FETCH/CLOSE)

```
BEGIN
```

```
  FOR rec IN (SELECT cot1, cot2 FROM bang) LOOP
```

```
    DBMS_OUTPUT.PUT_LINE(rec.cot1 || ' - ' || rec.cot2);
```

```
  END LOOP;
```

```
END;
```

```
/
```

-- Hoac dung cursor da khai bao:

```
FOR rec IN cur_ten LOOP
```

```
  DBMS_OUTPUT.PUT_LINE(rec.cot1);
```

```
END LOOP;
```

Các thuộc tính của Cursor

Thuộc tính	Kiểu trả về	Ý nghĩa
cur%FOUND	BOOLEAN	TRUE nếu FETCH vừa rồi lấy được hàng
cur%NOTFOUND	BOOLEAN	TRUE nếu FETCH vừa rồi KHÔNG lấy được hàng → dùng để thoát LOOP
cur%ROWCOUNT	NUMBER	Số hàng đã FETCH tính đến thời điểm hiện tại
cur%ISOPEN	BOOLEAN	TRUE nếu cursor đang mở

1.8. Thủ Tục Lưu Trữ (Stored Procedure)

Thủ tục là một khối PL/SQL được đặt tên, lưu trong database và có thể gọi lại nhiều lần. Thủ tục không trả về giá trị trực tiếp (dùng tham số OUT thay thế).

```
-- Cu phap tao thu tuc

CREATE OR REPLACE PROCEDURE ten_thu_tuc

    (p_tham_so_1 IN kieu_du_lieu,

    p_tham_so_2 OUT kieu_du_lieu,

    p_tham_so_3 IN OUT kieu_du_lieu)

IS

    -- Khai bao bien cuc bo

    v_tam NUMBER;

BEGIN

    -- Than thu tuc

    p_tham_so_2 := p_tham_so_1 * 2;

EXCEPTION

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('Loi: ' || SQLERRM);

END ten_thu_tuc;

/


-- Goi thu tuc

DECLARE

    v_out NUMBER;

BEGIN

    ten_thu_tuc(10, v_out);

    DBMS_OUTPUT.PUT_LINE('Ket qua: ' || v_out);

END;

/
```

Chế độ tham số	Ý nghĩa	Đặc điểm
IN	Tham số đầu vào - truyền giá trị vào thủ tục	Mặc định nếu không ghi. Chỉ đọc, không ghi
OUT	Tham số đầu ra - thủ tục gán giá trị để trả về	Có thể đọc và ghi. Cần biến để nhận
IN OUT	Vừa nhận vào vừa trả về - đọc và ghi được	Dùng khi cần vừa đọc giá trị cũ vừa ghi giá trị mới

1.9. Hàm Lưu Trữ (Stored Function)

Hàm giống thủ tục nhưng PHẢI trả về một giá trị bằng lệnh RETURN. Hàm có thể dùng trong câu lệnh SQL.

```
CREATE OR REPLACE FUNCTION ten_ham
    (p_tham_so IN kieu_du_lieu)
RETURN kieu_tra_ve
IS
    v_ket_qua kieu_tra_ve;
BEGIN
    -- Tính toán
    v_ket_qua := p_tham_so * 2;
    RETURN v_ket_qua;  -- Bat buoc co RETURN
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN NULL;
END ten_ham;
/
```

-- Goi ham trong SQL:

```
SELECT ten_ham(100) FROM DUAL;
```

-- Goi ham trong PL/SQL:

```
v_bien := ten_ham(100);
```

1.10. Trigger

Trigger là khối PL/SQL tự động thực thi khi có sự kiện DML (INSERT/UPDATE/DELETE) xảy ra trên bảng.

```
CREATE OR REPLACE TRIGGER ten_trigger

    BEFORE | AFTER          -- Khi nao chay: truoc hay sau su kien
    INSERT OR UPDATE OR DELETE -- Su kien kich hoat
    ON ten_bang              -- Bang theo doi
    [FOR EACH ROW]           -- Trigger hang: chay cho moi hang bi anh huong
                             -- Neu khong co: trigger bang (chay 1 lan)

DECLARE

    -- khai bao bien (neu can)

BEGIN

    -- :NEW.cot = gia tri MOI (sau INSERT/UPDATE)
    -- :OLD.cot = gia tri CU (truoc UPDATE/DELETE)

    IF INSERTING THEN

        :NEW.created_by := USER;
        :NEW.created_date := SYSDATE;
    END IF;

    IF INSERTING OR UPDATING THEN

        :NEW.modified_by := USER;
        :NEW.modified_date := SYSDATE;
    END IF;

END;
```

/

Thời điểm	:NEW	:OLD
INSERT	Giá trị vừa được thêm vào	NULL (không có giá trị cũ)
UPDATE	Giá trị mới sau khi cập nhật	Giá trị cũ trước khi cập nhật
DELETE	NULL (không có giá trị mới)	Giá trị bị xóa

1.11. Xử Lý Ngoại Lệ (Exception Handling)

Ngoại lệ	Khi nào xảy ra	Cách xử lý thường gặp
NO_DATA_FOUND	SELECT INTO không trả về hàng nào	Thông báo không tìm thấy, thêm mới nếu cần
TOO_MANY_ROWS	SELECT INTO trả về nhiều hơn 1 hàng	Dùng cursor thay thế
DUP_VAL_ON_INDEX	Vi phạm ràng buộc UNIQUE/PRIMARY KEY	Thông báo bị trùng, bỏ qua hoặc cập nhật
VALUE_ERROR	Lỗi chuyển đổi kiểu dữ liệu hoặc tràn số	Kiểm tra dữ liệu đầu vào
OTHERS	Bất tất cả ngoại lệ còn lại không được liệt kê	Dùng SQLERRM để lấy thông báo lỗi cụ thể

PHẦN 2: LƯỢC ĐỒ CƠ SỞ DỮ LIỆU - QUẢN LÝ KHÓA HỌC

2.1. Tổng Quan Các Bảng

Bảng	Mô tả	Cột chính
COURSE	Thông tin môn học	CourseNo (PK), Description, Cost, Prerequisite
CLASS	Thông tin lớp học cụ thể của môn	ClassID (PK), CourseNo (FK), ClassNo, StartDateTime, InstructorID (FK), Capacity
STUDENT	Thông tin sinh viên	StudentID (PK), FirstName, LastName, RegistrationDate
ENROLLMENT	Sinh viên đăng ký lớp học	StudentID (FK+PK), ClassID (FK+PK), EnrollDate, FinalGrade
INSTRUCTOR	Thông tin giáo viên	InstructorID (PK), FirstName, LastName
GRADE	Điểm sinh viên trong lớp học	StudentID (FK+PK), ClassID (FK+PK), Grade, Comments

2.2. Quan Hệ Giữa Các Bảng

- `CLASS.CourseNo` → `COURSE.CourseNo` (Lớp học thuộc môn học nào)
- `CLASS.InstructorID` → `INSTRUCTOR.InstructorID` (Lớp do giáo viên nào dạy)
- `ENROLLMENT.(StudentID,ClassID)` → `STUDENT.StudentID` và `CLASS.ClassID`
- `GRADE.(StudentID,ClassID)` → `ENROLLMENT.(StudentID,ClassID)` (Điểm ứng với mỗi đăng ký)
- `COURSE.Prerequisite` → `COURSE.CourseNo` (Môn tiên quyết - tự tham chiếu)

PHẦN 3: HƯỚNG DẪN CHI TIẾT TỪNG BÀI TẬP

Thiết lập trước khi bắt đầu: Mỗi lần mở phiên SQL*Plus mới cần chạy: `SET SERVEROUTPUT ON;` để thấy kết quả của `DBMS_OUTPUT.PUT_LINE`.

BÀI 1: Khởi Lệnh PL/SQL Cơ Bản - Transaction Và Savepoint

Câu 1a: Tạo bảng Cau1 với 2 cột ID (NUMBER) và NAME (VARCHAR2(20))

Phân tích: Đây là lệnh DDL thông thường - thực thi bên ngoài block PL/SQL.

```
CREATE TABLE Cau1 (
    ID      NUMBER,
    NAME    VARCHAR2 (20)
);
```

Câu 1b: Tạo sequence Cau1Seq với bước tăng là 5

```
CREATE SEQUENCE Cau1Seq
    START WITH 5
    INCREMENT BY 5;
```

Câu 1c–1j: Toàn bộ chương trình: biến, INSERT nhiều bước, Savepoint, Rollback

Phân tích: Đây là một block PL/SQL lớn kết hợp nhiều thao tác: SELECT INTO, INSERT, SAVEPOINT, ROLLBACK TO, sequence.

Chiến lược thực hiện từng bước:

- Bước c: Khai báo biến v_name (VARCHAR2) và v_id (NUMBER) trong phần DECLARE.
- Bước d: INSERT sinh viên đăng ký NHIỀU MÔN NHẤT - dùng subquery với MAX(COUNT).
- Bước e: INSERT sinh viên đăng ký ÍT MÔN NHẤT - tương tự, dùng MIN(COUNT).
- Bước f: INSERT giáo viên dạy NHIỀU LỚP NHẤT - kết bảng CLASS với INSTRUCTOR.
- Bước g: SELECT INTO - lấy ID của giáo viên vừa thêm vào biến v_id.
- Bước h: ROLLBACK TO Savepoint C - hủy thao tác của giáo viên nhiều lớp nhất.
- Bước i: INSERT lại giáo viên ÍT LỚP NHẤT - dùng v_id (ID đã rollback) thay vì sequence.
- Bước j: INSERT lại giáo viên nhiều lớp nhất với ID từ sequence.

Code hoàn chỉnh:

```
SET SERVEROUTPUT ON;

DECLARE
    v_name  VARCHAR2(50);
    v_id    NUMBER;
BEGIN
    -- [d] Them sinh vien dang ki nhieu mon nhat
    SELECT firstname || ' ' || lastname
    INTO    v_name
    FROM    student
    WHERE   studentid = (
        SELECT studentid FROM enrollment
        GROUP BY studentid
        HAVING COUNT(*) = (SELECT MAX(COUNT(*)) FROM enrollment GROUP BY
studentid)
        FETCH FIRST 1 ROWS ONLY
    );

    INSERT INTO Cau1 (ID, NAME)
    VALUES (Cau1Seq.NEXTVAL, v_name);
    SAVEPOINT sp_a;    -- Savepoint A

    -- [e] Them sinh vien dang ki it mon nhat
    SELECT firstname || ' ' || lastname
```

```
        INTO    v_name
      FROM      student
      WHERE     studentid = (
        SELECT  studentid FROM enrollment
        GROUP BY studentid
        HAVING  COUNT(*) = (SELECT MIN(COUNT(*)) FROM enrollment GROUP BY
studentid)
        FETCH  FIRST 1 ROWS ONLY
      );

      INSERT INTO Cau1 (ID, NAME)
      VALUES (Cau1Seq.NEXTVAL, v_name);
      SAVEPOINT sp_b;    -- Savepoint B

      -- [f] Them giao vien day nhieu lop nhat
      SELECT i.firstname || ' ' || i.lastname
      INTO    v_name
      FROM    instructor i
      WHERE   i.instructorid = (
        SELECT instructorid FROM class
        GROUP BY instructorid
        HAVING COUNT(*) = (SELECT MAX(COUNT(*)) FROM class GROUP BY
instructorid)
        FETCH FIRST 1 ROWS ONLY
      );

      INSERT INTO Cau1 (ID, NAME)
      VALUES (Cau1Seq.NEXTVAL, v_name);
      SAVEPOINT sp_c;    -- Savepoint C

      -- [g] SELECT INTO: lay ID cua giao vien vua them vao bien v_id
      SELECT ID INTO v_id
      FROM    Cau1
      WHERE   NAME = v_name;

      DBMS_OUTPUT.PUT_LINE('ID giao vien nhieu lop: ' || v_id);

      -- [h] Rollback giao vien nhieu lop (ve Savepoint B)
```



```
ROLLBACK TO sp_b;

-- [i] Them giao vien it lop nhat, dung v_id da luu
SELECT i.firstname || ' ' || i.lastname
INTO   v_name
FROM   instructor i
WHERE  i.instructorid = (
        SELECT instructorid FROM class
        GROUP BY instructorid
        HAVING COUNT(*) = (SELECT MIN(COUNT(*)) FROM class GROUP BY
instructorid)
        FETCH FIRST 1 ROWS ONLY
    );

INSERT INTO Cau1 (ID, NAME)
VALUES (v_id, v_name);    -- Dung v_id (khong phai sequence)

-- [j] Them lai giao vien nhieu lop, dung sequence
SELECT i.firstname || ' ' || i.lastname
INTO   v_name
FROM   instructor i
WHERE  i.instructorid = (
        SELECT instructorid FROM class
        GROUP BY instructorid
        HAVING COUNT(*) = (SELECT MAX(COUNT(*)) FROM class GROUP BY
instructorid)
        FETCH FIRST 1 ROWS ONLY
    );

INSERT INTO Cau1 (ID, NAME)
VALUES (Cau1Seq.NEXTVAL, v_name);    -- Dung sequence

COMMIT;

DBMS_OUTPUT.PUT_LINE('Hoan tat! Kiem tra: SELECT * FROM Cau1;');

EXCEPTION

WHEN NO_DATA_FOUND THEN

    DBMS_OUTPUT.PUT_LINE('Loi: Khong tim thay du lieu!');
```

```

        ROLLBACK;

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('Loi: ' || SQLERRM);

        ROLLBACK;

    END;

/

```

Lưu ý: FETCH FIRST 1 ROWS ONLY chỉ dùng được từ Oracle 12c trở lên. Nếu dùng phiên bản cũ hơn, thay bằng ROWNUM = 1 trong subquery bên trong.

Câu 2: Nhập mã sinh viên - kiểm tra tồn tại → hiển thị thông tin; không tồn tại → thêm sinh viên mới

Phân tích: Dùng SELECT INTO trong khối BEGIN. Nếu NO_DATA_FOUND thì bắt exception và INSERT mới.

```

SET SERVEROUTPUT ON;

DECLARE

    v_sid      NUMBER          := &ma_sinh_vien;
    v_fname    VARCHAR2(25)    := '&ho_sinh_vien';
    v_lname    VARCHAR2(25)    := '&ten_sinh_vien';
    v_addr     VARCHAR2(50)    := '&dia_chi';
    v_found    VARCHAR2(50);
    v_classes  NUMBER;

BEGIN

    -- Thu tìm sinh viên theo ma vừa nhập
    SELECT firstname || ' ' || lastname
    INTO    v_found
    FROM    student
    WHERE   studentid = v_sid;

    -- Nếu tìm thấy: đếm số lớp đang học
    SELECT COUNT(*)
    INTO    v_classes
    FROM    enrollment
    WHERE   studentid = v_sid;

    DBMS_OUTPUT.PUT_LINE('Ho ten: ' || v_found);
    DBMS_OUTPUT.PUT_LINE('Số lớp đang học: ' || v_classes);

```

```

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        -- Sinh vien chua ton tai: them moi
        DBMS_OUTPUT.PUT_LINE('Sinh vien chua ton tai. Dang them moi...');
        INSERT INTO student (studentid, firstname, lastname, address,
                             registrationdate, createdby, createddate,
                             modifiedby, modifieddate)
        VALUES (v_sid, v_fname, v_lname, v_addr,
                SYSDATE, USER, SYSDATE, USER, SYSDATE);
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Da them sinh vien moi: ' || v_fname || ' ' ||
v_lname);
END;
/

```

*Lưu ý: Các giá trị &biến sẽ hiện hộp thoại nhập liệu trong SQL*Plus. Nếu nhập chuỗi cho &ho_sinh_vien và &ten_sinh_vien thì Oracle sẽ tự bỏ dấu nháy đơn.*

BÀI 2: Các Cấu Trúc Điều Khiển - IF và CASE

Câu 1: Nhập mã giáo viên → đếm số lớp đang dạy → IF ≥ 5 : thông báo nghỉ ngơi, ELSE in số lớp

Phân tích: Dùng SELECT INTO đếm COUNT, sau đó dùng IF/ELSE để phân nhánh kết quả.

```

SET SERVEROUTPUT ON;

DECLARE
    v_instructor_id  NUMBER := &ma_giao_vien;
    v_so_lop          NUMBER;
BEGIN
    -- Dem so lop giao vien dang day
    SELECT COUNT(*)
    INTO    v_so_lop

```

```

FROM class
WHERE instructorid = v_instructor_id;

-- Phan nhanh theo ket qua
IF v_so_lop >= 5 THEN
    DBMS_OUTPUT.PUT_LINE('Giao vien nay nen nghi ngoi!');
ELSE
    DBMS_OUTPUT.PUT_LINE('So lop giao vien dang day: ' || v_so_lop);
END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Khong tim thay giao vien co ma: ' ||
v_instructor_id);
END;
/

```

Câu 2: Nhập mã sinh viên + mã lớp → in điểm chữ bằng CASE; xử lý lỗi khi mã không tồn tại

Phân tích: SELECT INTO lấy FinalGrade từ ENROLLMENT. Dùng CASE WHEN để quy đổi điểm số → điểm chữ. Xử lý NO_DATA_FOUND cho cả 2 trường hợp sai mã.

```

SET SERVEROUTPUT ON;

DECLARE
    v_sid    NUMBER := &ma_sinh_vien;
    v_cid    NUMBER := &ma_lop;
    v_score  NUMBER;
    v_grade  VARCHAR2(2);
    v_check  NUMBER;
BEGIN
    -- Kiem tra sinh vien ton tai
    SELECT COUNT(*) INTO v_check
    FROM student WHERE studentid = v_sid;

```

```
        IF v_check = 0 THEN
            DBMS_OUTPUT.PUT_LINE('Loi: Ma sinh vien ' || v_sid || ' khong ton
tai!');
            RETURN;
        END IF;

        -- Kiem tra lop ton tai
        SELECT COUNT(*) INTO v_check
        FROM   class WHERE classid = v_cid;
        IF v_check = 0 THEN
            DBMS_OUTPUT.PUT_LINE('Loi: Ma lop ' || v_cid || ' khong ton tai!');
            RETURN;
        END IF;

        -- Lay diem cua sinh vien trong lop
        SELECT finalgrade
        INTO   v_score
        FROM   enrollment
        WHERE  studentid = v_sid AND classid = v_cid;

        -- Quy doi diem so sang diem chu bang CASE
        CASE
            WHEN v_score >= 90 THEN v_grade := 'A';
            WHEN v_score >= 80 THEN v_grade := 'B';
            WHEN v_score >= 70 THEN v_grade := 'C';
            WHEN v_score >= 50 THEN v_grade := 'D';
            ELSE v_grade := 'F';
        END CASE;

        DBMS_OUTPUT.PUT_LINE('Diem so: ' || v_score || ' -> Diem chu: ' ||
v_grade);

    EXCEPTION

        WHEN NO_DATA_FOUND THEN

            DBMS_OUTPUT.PUT_LINE('Sinh vien chua dang ky lop nay hoac chua co
diem!');
        END;
    /
```

Lưu ý: Lệnh RETURN trong PL/SQL block (không phải hàm) sẽ kết thúc ngay block đó. Đây là cách thoát sớm khi kiểm tra điều kiện đầu vào không hợp lệ.

BÀI 3: Cursor - Duyệt Tập Kết Quả

Yêu cầu: In ra thông tin tất cả môn học, mỗi môn in ra các lớp học kèm số sinh viên đăng ký.

Định dạng mong muốn:

10 DP Overview

Lop: 2 co so luong sinh vien dang ki: 1

20 Intro to Computers

Lop: 2 co so luong sinh vien dang ki: 3

Lop: 4 co so luong sinh vien dang ki: 2

...

Hướng dẫn - Bài 3: Cursor lồng nhau

Phân tích: Cần 2 cursor lồng nhau: cursor ngoài duyệt từng môn học, cursor trong nhận đối số CourseNo để lấy lớp học và số SV đăng ký của môn đó.

Cấu trúc tổng quát:

- Cursor 1 (cur_course): SELECT CourseNo, Description FROM course ORDER BY CourseNo
- Cursor 2 (cur_class có đối số p_courseno): SELECT c.ClassNo, COUNT(e.StudentID) FROM class c LEFT JOIN enrollment e... WHERE c.CourseNo = p_courseno GROUP BY c.ClassNo

Code hoàn chỉnh:

```
SET SERVEROUTPUT ON;

DECLARE

    -- Cursor 1: Duyệt từng môn học
    CURSOR cur_course IS
        SELECT courseno, description
        FROM   course
        ORDER BY courseno;
```

```
-- Cursor 2: Lay lop hoc cua mot mon (co doi so)
CURSOR cur_class (p_courseno NUMBER) IS
    SELECT c.classno,
           COUNT(e.studentid) AS so_sv
    FROM   class c
    LEFT   JOIN enrollment e ON c.classid = e.classid
    WHERE  c.courseno = p_courseno
    GROUP  BY c.classno
    ORDER  BY c.classno;

v_courseno  course.courseno%TYPE;
v_desc      course.description%TYPE;
v_classno   class.classno%TYPE;
v_count     NUMBER;
BEGIN
    -- Duyệt cursor ngoài: từng môn học
    OPEN cur_course;
    LOOP
        FETCH cur_course INTO v_courseno, v_desc;
        EXIT WHEN cur_course%NOTFOUND;

        -- In tên môn học
        DBMS_OUTPUT.PUT_LINE(v_courseno || ' ' || v_desc);

        -- Mở cursor trong với đối số là mã môn học hiện tại
        OPEN cur_class(v_courseno);
        LOOP
            FETCH cur_class INTO v_classno, v_count;
            EXIT WHEN cur_class%NOTFOUND;
            DBMS_OUTPUT.PUT_LINE('Lop: ' || v_classno ||
                                ' có số lượng sinh viên đang ki: ' ||
v_count);
        END LOOP;
        CLOSE cur_class;

    END LOOP;
    CLOSE cur_course;

EXCEPTION
    WHEN OTHERS THEN
        IF cur_course%ISOPEN THEN CLOSE cur_course; END IF;
```



```

        IF cur_class%ISOPEN THEN CLOSE cur_class; END IF;
        DBMS_OUTPUT.PUT_LINE('Loi: ' || SQLERRM);
    END;
/

```

Lưu ý: Cursor có đối số phải được OPEN với giá trị cụ thể: OPEN cur_class(v_courseno). Cursor trong (cur_class) cần được CLOSE sau mỗi vòng lặp ngoài trước khi OPEN lại cho môn tiếp theo.

BÀI 4: Thủ Tục (Procedure) Và Hàm (Function)

Câu 4.1a: Thủ tục find_sname - nhận mã SV, trả về FirstName và LastName

Phân tích: Tham số IN nhận mã sinh viên. Hai tham số OUT trả về họ và tên. Dùng SELECT INTO để lấy dữ liệu.

```

CREATE OR REPLACE PROCEDURE find_sname
(
    i_student_id IN student.studentid%TYPE,
    o_first_name OUT student.firstname%TYPE,
    o_last_name OUT student.lastname%TYPE)
IS
BEGIN
    SELECT firstname, lastname
    INTO o_first_name, o_last_name
    FROM student
    WHERE studentid = i_student_id;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        o_first_name := NULL;
        o_last_name := NULL;
        DBMS_OUTPUT.PUT_LINE('Không tìm thấy sinh viên ID: ' ||
i_student_id);
END find_sname;
/

```

Câu 4.1b: Thủ tục print_student_name nhận mã SV, gọi find_sname và in tên ra màn hình

Phân tích: Thủ tục này GỌI thủ tục find_sname đã viết ở trên - thể hiện cách tái sử dụng code.

```
CREATE OR REPLACE PROCEDURE print_student_name
    (i_student_id IN student.studentid%TYPE)
IS
    v_first student.firstname%TYPE;
    v_last student.lastname%TYPE;
BEGIN
    -- Goi thu tuc find_sname da co san
    find_sname(i_student_id, v_first, v_last);

    IF v_first IS NOT NULL OR v_last IS NOT NULL THEN
        DBMS_OUTPUT.PUT_LINE('Ho ten sinh vien: ' || v_first || ' ' ||
v_last);
    END IF;
END print_student_name;
/

-- Goi thu tuc de kiem tra:
BEGIN
    print_student_name(101);
END;
/
```

Câu 4.2: Thủ tục Discount - giảm 5% cho môn học có hơn 15 sinh viên đăng ký

Phân tích: Dùng cursor FOR LOOP duyệt các môn học thỏa điều kiện. Với mỗi môn: UPDATE giảm giá và in tên môn.

```
CREATE OR REPLACE PROCEDURE Discount
IS
BEGIN
    FOR rec IN (
        SELECT c.courseno, c.description, c.cost
        FROM course c
```

```

        WHERE (SELECT COUNT(*) FROM enrollment e
                JOIN class cl ON e.classid = cl.classid
                WHERE cl.courseno = c.courseno) > 15
    ) LOOP
        -- Giảm giá 5%
        UPDATE course
        SET     cost = cost * 0.95
        WHERE   courseno = rec.courseno;

        DBMS_OUTPUT.PUT_LINE('Đã giảm giá môn học: ' || rec.description
                               || ' | Giá cũ: ' || rec.cost
                               || ' | Giá mới: ' || ROUND(rec.cost * 0.95, 2));
    END LOOP;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Hoàn tất giảm giá.');
```

EXCEPTION

```

    WHEN OTHERS THEN
        ROLLBACK;

        DBMS_OUTPUT.PUT_LINE('Lỗi: ' || SQLERRM);
END Discount;
/

-- Gọi thủ tục:
BEGIN Discount; END;
/
```

Câu 4.3: Hàm Total_cost_for_student - trả về tổng chi phí một sinh viên phải trả; NULL nếu không tồn tại

*Phân tích: Hàm kết bảng ENROLLMENT → CLASS → COURSE để tính SUM(cost).
Xử lý NO_DATA_FOUND trả về NULL.*

```

CREATE OR REPLACE FUNCTION Total_cost_for_student
    (p_student_id IN student.studentid%TYPE)
RETURN NUMBER
```

```
IS
    v_total    NUMBER;
    v_check    NUMBER;
BEGIN
    -- Kiểm tra sinh viên có tồn tại không
    SELECT COUNT(*) INTO v_check
    FROM    student WHERE studentid = p_student_id;

    IF v_check = 0 THEN
        RETURN NULL;    -- Sinh viên không tồn tại
    END IF;

    -- Tính tổng chi phí: sum(cost của từng môn đã đăng ký)
    SELECT NVL(SUM(co.cost), 0)
    INTO    v_total
    FROM    enrollment e
    JOIN    class      cl ON e.classid = cl.classid
    JOIN    course      co ON cl.courseno = co.courseno
    WHERE   e.studentid = p_student_id;

    RETURN v_total;

EXCEPTION
    WHEN OTHERS THEN
        RETURN NULL;
END Total_cost_for_student;
/

-- Gọi hàm để kiểm tra:
SELECT Total_cost_for_student(101) AS "Tổng chi phí" FROM DUAL;

-- Hoặc trong PL/SQL:
BEGIN
    DBMS_OUTPUT.PUT_LINE('Tổng chi phí: ' || Total_cost_for_student(101));
END;
/
```


BÀI 5: Trigger

Câu 5.1: Trigger tự động điền created_by, created_date, modified_by, modified_date cho tất cả các bảng

Phân tích: Cần tạo trigger BEFORE INSERT OR UPDATE cho từng bảng. Trigger sẽ tự gán USER và SYSDATE. Viết cho 1 bảng mẫu, sau đó sao chép tương tự cho các bảng còn lại.

Trigger mẫu cho bảng COURSE:

```
CREATE OR REPLACE TRIGGER trg_course_audit
  BEFORE INSERT OR UPDATE ON course
  FOR EACH ROW
BEGIN
  IF INSERTING THEN
    :NEW.created_by := USER;
    :NEW.created_date := SYSDATE;
  END IF;
  -- Luôn cập nhật modified (cả khi INSERT lẫn UPDATE)
  :NEW.modified_by := USER;
  :NEW.modified_date := SYSDATE;
END trg_course_audit;
/
```

Trigger tương tự cho bảng CLASS:

```
CREATE OR REPLACE TRIGGER trg_class_audit
  BEFORE INSERT OR UPDATE ON class
  FOR EACH ROW
BEGIN
  IF INSERTING THEN
    :NEW.created_by := USER;
    :NEW.created_date := SYSDATE;
  END IF;
  :NEW.modified_by := USER;
  :NEW.modified_date := SYSDATE;
END trg_class_audit;
```

Tương tự cho STUDENT, ENROLLMENT, INSTRUCTOR, GRADE - chỉ cần đổi tên trigger và tên bảng:

```
-- STUDENT
CREATE OR REPLACE TRIGGER trg_student_audit
    BEFORE INSERT OR UPDATE ON student FOR EACH ROW
BEGIN
    IF INSERTING THEN :NEW.created_by:=USER; :NEW.created_date:=SYSDATE; END
    IF;

    :NEW.modified_by:=USER; :NEW.modified_date:=SYSDATE;
END; /

-- ENROLLMENT
CREATE OR REPLACE TRIGGER trg_enrollment_audit
    BEFORE INSERT OR UPDATE ON enrollment FOR EACH ROW
BEGIN
    IF INSERTING THEN :NEW.created_by:=USER; :NEW.created_date:=SYSDATE; END
    IF;

    :NEW.modified_by:=USER; :NEW.modified_date:=SYSDATE;
END; /

-- INSTRUCTOR
CREATE OR REPLACE TRIGGER trg_instructor_audit
    BEFORE INSERT OR UPDATE ON instructor FOR EACH ROW
BEGIN
    IF INSERTING THEN :NEW.created_by:=USER; :NEW.created_date:=SYSDATE; END
    IF;

    :NEW.modified_by:=USER; :NEW.modified_date:=SYSDATE;
END; /

-- GRADE
CREATE OR REPLACE TRIGGER trg_grade_audit
    BEFORE INSERT OR UPDATE ON grade FOR EACH ROW
BEGIN
    IF INSERTING THEN :NEW.created_by:=USER; :NEW.created_date:=SYSDATE; END
    IF;

    :NEW.modified_by:=USER; :NEW.modified_date:=SYSDATE;
END; /
```

Giải thích: Dùng BEFORE INSERT OR UPDATE để trigger chạy TRƯỚC khi Oracle ghi xuống đĩa đây là điều bắt buộc vì ta cần gán giá trị cho :NEW trước khi ghi. Nếu dùng AFTER thì :NEW đã được ghi rồi, không thể thay đổi nữa.

Câu 5.2: Trigger kiểm tra: mỗi sinh viên không được đăng ký quá 3 lớp học

Phân tích: Trigger BEFORE INSERT trên bảng ENROLLMENT. Đếm số lớp hiện tại của sinh viên, nếu đã ≥ 3 thì RAISE_APPLICATION_ERROR để từ chối INSERT.

```
CREATE OR REPLACE TRIGGER trg_max_enrollment
  BEFORE INSERT ON enrollment
  FOR EACH ROW
DECLARE
  v_so_lop  NUMBER;
BEGIN
  -- Dem so lop sinh vien nay dang dang ky
  SELECT COUNT(*)
  INTO   v_so_lop
  FROM   enrollment
  WHERE  studentid = :NEW.studentid;

  -- Neu da co 3 lop tro len thi tu choi
  IF v_so_lop >= 3 THEN
    RAISE_APPLICATION_ERROR(
      -20001,
      'Sinh vien ' || :NEW.studentid ||
      ' da dang ky du 3 lop! Khong the dang ky them.'
    );
  END IF;
END trg_max_enrollment;
/

-- Kiem tra trigger:
-- Gia su sinh vien 101 da co 3 lop, thu them lop thu 4:
INSERT INTO enrollment (studentid, classid, enrolldate, createdby,
```



```

        createddate, modifiedby, modifieddate)
VALUES (101, 999, SYSDATE, USER, SYSDATE, USER, SYSDATE);
-- -> Oracle sẽ báo lỗi ORA-20001

```

Giải thích: RAISE_APPLICATION_ERROR(-20001, 'thông báo') là cách chuẩn để tạo lỗi tùy chỉnh trong Oracle. Mã lỗi phải nằm trong khoảng -20000 đến -20999 (dành riêng cho ứng dụng người dùng).

Lưu ý: Trigger này kiểm tra TRƯỚC khi INSERT, nên :NEW.studentid là giá trị sắp được thêm vào. COUNT() đếm các hàng hiện có trong bảng (chưa kể hàng đang INSERT).*

PHẦN 4: BẢNG TRA CỨU NHANH VÀ CÁC LỖI THƯỜNG GẶP

4.1. Bảng Tổng Hợp Kỹ Thuật Theo Từng Bài

Bài	Kỹ thuật chính cần nhớ	Điểm mấu chốt
Bài 1	Block PL/SQL, SELECT INTO, INSERT, SAVEPOINT, ROLLBACK TO, Sequence	SAVEPOINT không hủy thay đổi - ROLLBACK TO mới hủy từ điểm đó trở đi
Bài 2	IF/ELSIF/ELSE, CASE WHEN, RETURN sớm, xử lý lỗi đầu vào	Kiểm tra dữ liệu đầu vào trước khi xử lý chính
Bài 3	Cursor tường minh, cursor có đối số, cursor lồng nhau, %NOTFOUND	Cursor trong phải CLOSE rồi mới OPEN lại cho vòng lặp tiếp theo
Bài 4	Procedure IN/OUT, gọi procedure từ procedure, Function RETURN, Cursor FOR LOOP	Hàm PHẢI có RETURN. Thủ tục dùng tham số OUT để trả giá trị
Bài 5	BEFORE Trigger, :NEW/:OLD, INSERTING/UPDATING, RAISE_APPLICATION_ERROR	Dùng BEFORE (không phải AFTER) để gán :NEW. Mã lỗi tùy chỉnh từ -20000 đến -20999

4.2. Các Lỗi Phổ Biến Trong PL/SQL

Lỗi	Nguyên nhân	Cách sửa
ORA-01403: no data found	SELECT INTO không trả về hàng nào	Bắt EXCEPTION WHEN NO_DATA_FOUND
ORA-01422: exact fetch returns more than requested	SELECT INTO trả về nhiều hơn 1 hàng	Dùng cursor thay SELECT INTO, hoặc thêm ROWNUM=1
ORA-06503: PL/SQL: Function returned without value	Hàm (Function) thiếu lệnh RETURN trong một nhánh	Đảm bảo mọi nhánh IF/EXCEPTION đều có RETURN
ORA-04082: NEW or OLD references not allowed	Dùng :NEW/:OLD trong trigger mức bảng (thiếu FOR EACH ROW)	Thêm FOR EACH ROW vào khai báo trigger
ORA-04091: table is mutating	Trigger đọc/ghi chính bảng đang bị thay đổi	Dùng compound trigger hoặc autonomous transaction
PLS-00201: identifier must be declared	Dùng biến chưa khai báo hoặc sai tên	Kiểm tra chính tả biến trong phần DECLARE
Cursor vẫn mở — ORA-06511	Mở cursor đã mở mà không đóng trước	Kiểm tra %ISOPEN trước khi OPEN, CLOSE trong phần EXCEPTION

4.3. Checklist Thiết Lập Môi Trường

Lệnh	Mục đích
SET SERVEROUTPUT ON;	Bật hiển thị kết quả DBMS_OUTPUT - phải chạy mỗi phiên mới
SET SERVEROUTPUT ON SIZE 1000000;	Tăng buffer nếu output nhiều (tránh lỗi buffer overflow)
SHOW ERRORS;	Xem chi tiết lỗi compile sau khi tạo procedure/function/trigger
SELECT * FROM user_errors;	Xem danh sách tất cả lỗi compile của các đối tượng
SELECT object_name, status FROM user_objects WHERE object_type='TRIGGER';	Kiểm tra trạng thái (VALID/INVALID) của các trigger
ALTER TRIGGER ten_trigger DISABLE;	Tạm tắt trigger (để test dữ liệu không bị trigger chặn)
ALTER TRIGGER ten_trigger ENABLE;	Bật lại trigger

Good luck!